

Polymorphism in C++ is a fundamental concept in object-oriented programming that allows objects to be treated as instances of their parent class, enabling a single interface to represent different underlying forms (data types). There are two primary types of polymorphism in C++:

1. Compile-Time Polymorphism

Compile-time polymorphism is resolved during the compilation of the program. It is achieved through:

- **Function Overloading:** Multiple functions can have the same name but different parameters (type or number).
- **Operator Overloading:** Operators can be redefined to work with user-defined types.

Example of Function Overloading

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2using namespace std;
3
4class Overload {
5public:
6    void print(int num) {
7        cout << "Printing integer: " << num << endl;
8    }
9
10   void print(double num) {
11       cout << "Printing double: " << num << endl;
12   }
13
14   void print(string text) {
15       cout << "Printing string: " << text << endl;
16   }
17};
18
19int main() {
20    Overload obj;
21    obj.print(5);           // Calls print(int)
22    obj.print(10.5);       // Calls print(double)
23    obj.print("Hello");    // Calls print(string)
24    return 0;
25}
```

Output:

```
VerifyOpen In EditorRunCopy code
1Printing integer: 5
2Printing double: 10.5
3Printing string: Hello
```

Example of Operator Overloading

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2using namespace std;
3
4class StringConcat {
5private:
```

```

6    string str;
7public:
8    StringConcat(string s) : str(s) {}
9
10   // Overloading the + operator
11   StringConcat operator+(const StringConcat& obj) {
12       return StringConcat(this->str + obj.str);
13   }
14
15   void display() {
16       cout << "Concatenated String: " << str << endl;
17   }
18};
19
20int main() {
21    StringConcat str1("Hello");
22    StringConcat str2(" World");
23    StringConcat result = str1 + str2; // Uses overloaded +
24    result.display();
25    return 0;
26}

```

Output:

VerifyOpen In EditorRunCopy code
1Concatenated String: Hello World

2. Runtime Polymorphism

Runtime polymorphism is resolved during the execution of the program. It is achieved through:

- **Function Overriding:** A derived class provides a specific implementation of a function that is already defined in its base class.
- **Virtual Functions:** Functions declared with the `virtual` keyword in the base class.

Example of Function Overriding

```

cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2using namespace std;
3
4class Car {
5public:
6    virtual void displayBrand() {
7        cout << "This is a generic car." << endl;
8    }
9};
10
11class Ford : public Car {
12public:
13    void displayBrand() override {
14        cout << "This is a Ford car." << endl;
15    }
16};
17
18class Toyota : public Car {
19public:
20    void displayBrand() override {
21        cout << "This is a Toyota car." << endl;
22    }

```

```

23};
24
25int main() {
26    Car* car1 = new Ford();
27    Car* car2 = new Toyota();
28
29    car1->displayBrand(); // Calls Ford's displayBrand
30    car2->displayBrand(); // Calls Toyota's displayBrand
31
32    delete car1;
33    delete car2;
34    return 0;
35}

```

Output:

VerifyOpen In EditorRunCopy code
1This is a Ford car.
2This is a Toyota car.

Key Differences Between Compile-Time and Runtime Polymorphism

Feature	Compile-Time Polymorphism	Runtime Polymorphism
Binding	Static binding	Dynamic binding
Resolution	Resolved at compile time	Resolved at runtime
Flexibility	Less flexible	More flexible
Implementation	Function overloading, operator overloading	Function overriding, virtual functions
Performance	Faster execution	Slower execution

Conclusion

Polymorphism enhances the flexibility and reusability of code in C++. By allowing functions to operate on different data types and enabling a single interface to represent different underlying forms, it simplifies code management and improves maintainability. Understanding and implementing both compile-time and runtime polymorphism is crucial for effective C++ programming.

Understanding Friend Functions in C++

Friend functions in C++ are special functions that are not members of a class but have access to its private and protected members. This feature allows for more flexible interactions between classes while maintaining encapsulation.

Key Features of Friend Functions

- **Access Control:** Friend functions can access private and protected members of the class that declares them as friends.
- **Non-Member Functions:** They are not part of the class's scope and are called like regular functions.
- **Declaration:** A friend function is declared using the `friend` keyword inside the class definition.

Syntax

```
cpp
VerifyOpen In EditorRunCopy code
1class ClassName {
2private:
3    // Private members
4public:
5    // Public members
6    friend return_type functionName(parameters); // Declaration of friend
function
7};
```

Examples of Friend Functions

Example 1: Basic Friend Function

This example demonstrates a simple friend function that calculates the average of two private members.

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2using namespace std;
3
4class AverageCalculator {
5private:
6    int num1, num2;
7public:
8    AverageCalculator(int a, int b) : num1(a), num2(b) {}
9
10    // Friend function to calculate average
11    friend float calculateAverage(AverageCalculator obj);
12};
13
14// Definition of friend function
15float calculateAverage(AverageCalculator obj) {
16    return (float)(obj.num1 + obj.num2) / 2;
17}
18
19int main() {
20    AverageCalculator obj(10, 20);
21    cout << "Average: " << calculateAverage(obj) << endl;
22    return 0;
23}
```

Output:

```
VerifyOpen In EditorRunCopy code
1Average: 15
```

Example 2: Friend Function for Matrix Addition

This example shows how a friend function can be used to perform operations on private members of two different classes.

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2using namespace std;
3
```

```

4class Matrix {
5private:
6    int mat[2][2];
7public:
8    Matrix() {
9        cout << "Enter 4 matrix elements: ";
10        for (int i = 0; i < 2; ++i)
11            for (int j = 0; j < 2; ++j)
12                cin >> mat[i][j];
13    }
14
15    // Friend function for matrix addition
16    friend Matrix addMatrices(Matrix m1, Matrix m2);
17
18    void display() {
19        cout << "Matrix:" << endl;
20        for (int i = 0; i < 2; ++i) {
21            for (int j = 0; j < 2; ++j)
22                cout << mat[i][j] << " ";
23            cout << endl;
24        }
25    }
26};
27
28// Definition of friend function
29Matrix addMatrices(Matrix m1, Matrix m2) {
30    Matrix result;
31    for (int i = 0; i < 2; ++i)
32        for (int j = 0; j < 2; ++j)
33            result.mat[i][j] = m1.mat[i][j] + m2.mat[i][j];
34    return result;
35}
36
37int main() {
38    Matrix m1, m2;
39    Matrix sum = addMatrices(m1, m2);
40    sum.display();
41    return 0;
42}

```

Output:

VerifyOpen In EditorRunCopy code

```

1Matrix:
25 7
39 11

```

Example 3: Friend Function for Incrementing a Value

This example illustrates how a friend function can modify a private member of a class.

```

cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2using namespace std;
3
4class Number {
5private:
6    int value;
7public:
8    Number(int v) : value(v) {}
9
10    // Friend function for increment

```

```

11    friend Number increment(Number num);
12
13    void display() {
14        cout << "Value: " << value << endl;
15    }
16};
17
18// Definition of friend function
19Number increment(Number num) {
20    num.value++;
21    return num;
22}
23
24int main() {
25    Number num(5);
26    num.display();
27    num = increment(num);
28    num.display();
29    return 0;
30}

```

Output:

VerifyOpen In EditorRunCopy code

1Value: 5

2Value: 6

Advantages of Friend Functions

- **Access to Private Members:** They allow access to private and protected members of a class, which can be useful for operator overloading and inter-class communication.
- **Encapsulation:** They help maintain encapsulation while allowing specific functions to access class internals.
- **Flexibility:** They can be used to implement functionality that doesn't logically belong to a class.

Disadvantages of Friend Functions

- **Tight Coupling:** Excessive use can lead to tight coupling between classes, making the code harder to maintain.
- **Not Polymorphic:** Friend functions do not support polymorphism, which can limit their use in certain scenarios.

Conclusion

Friend functions are a powerful feature in C++ that allow for flexible interactions between classes while maintaining encapsulation. They should be used judiciously to avoid tight coupling and maintain code clarity.

Explanation of Constant Functions in C++

In C++, the `const` keyword is used to define constant functions, which are member functions that do not modify the object for which they are called. This is particularly useful for ensuring that certain functions can be called on constant objects without risking any changes to their state.

Key Points about Constant Functions

- **Declaration:** A constant member function is declared by appending the `const` keyword to the function signature, after the parameter list.
- **Usage:** Constant functions can be called on both constant and non-constant objects, while non-constant functions can only be called on non-constant objects.
- **Purpose:** They help prevent accidental modifications to the object's data members, enhancing code safety and readability.

Example of Constant Functions

Here's a simple example demonstrating the use of constant functions in a C++ class:

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2using namespace std;
3
4class Sample {
5private:
6    int value;
7
8public:
9    // Constructor to initialize value
10    Sample(int x = 0) : value(x) {}
11
12    // Constant member function
13    int getValue() const {
14        return value; // This function does not modify the object
15    }
16
17    // Non-constant member function
18    void setValue(int x) {
19        value = x; // This function modifies the object
20    }
21};
22
23int main() {
24    const Sample s(20); // Constant object
25    Sample s1(2);       // Non-constant object
26
27    // Calling constant function on both constant and non-constant objects
28    cout << "Value from constant object s: " << s.getValue() << endl; // OK
29    cout << "Value from non-constant object s1: " << s1.getValue() << endl; //
OK
30
31    // Calling non-constant function on non-constant object
32    s1.setValue(10); // OK
33    cout << "Updated value from non-constant object s1: " << s1.getValue() <<
endl; // OK
34
35    // Uncommenting the following line will cause a compile-time error
36    // s.setValue(30); // Error: cannot call non-const function on const
object
37
38    return 0;
39}
```

Explanation of the Example

- **Class Definition:** The `Sample` class has a private member `value` and two member functions: `getValue()` and `setValue()`.
- **Constant Function:** The `getValue()` function is declared as `const`, meaning it guarantees not to modify any member variables of the class.
- **Non-Constant Function:** The `setValue()` function modifies the `value` member, so it cannot be called on a constant object.
- **Main Function:** In the `main()` function, we create both constant and non-constant objects of the `Sample` class and demonstrate the usage of both types of member functions.

Conclusion

Using constant functions in C++ is a best practice that helps maintain the integrity of objects by preventing unintended modifications. By declaring member functions as `const`, you can ensure that they are safe to call on constant instances of your classes.